Experiment 1: AI-Assisted Boolean Expression Simplifier

Part B: GenAI-Assisted Design | CO Coverage: CO1, CO2 | EDA Tool: Logisim | AI Tool: ChatGPT / Gemini

Objective

To formulate, simplify, and verify a 4-variable Boolean logic function using Generative AI (Karnaugh Map minimization), implement both the unsimplified canonical Sum of Products (SOP) circuit and the minimized SOP circuit in the Logisim digital logic simulator, and verify functional equivalence across all input combinations.

Theory & Problem Statement

In digital logic design, implementing Boolean expressions directly in their canonical standard form results in excessive gate count, high propagation delay, increased dynamic power dissipation, and substantial silicon area consumption. Karnaugh Maps (K-maps) provide an established graphical minimization method that eliminates redundant literals by grouping adjacent minterm cells that differ by only a single literal (Gray code property).

The assigned 4-variable Boolean function in Sum-of-Minterms notation is:

$$F(A, B, C, D) = \sum m(0, 2, 8, 10, 12, 14)$$

Where A is the Most Significant Bit (MSB) and D is the Least Significant Bit (LSB). The canonical unsimplified Sum of Products (SOP) expression is:

$$F = A'B'C'D' + A'B'CD' + AB'C'D' + AB'CD' + ABC'D' + ABCD'$$

GenAI Prompt & Interaction

Generative AI was used to automate K-map cell grouping, identify essential prime implicants, and derive the minimal SOP expression.

```
User Prompt:
"Simplify the 4-variable Boolean expression F(A,B,C,D) = Σ m(0, 2, 8, 10, 12, 14) using a Karnaugh map and
provide the minimized SOP expression"
```

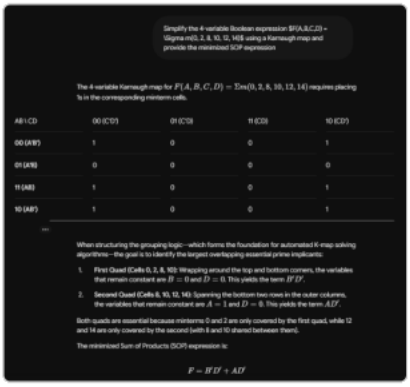


Figure 1.1: GenAI response detailing K-map distribution, Quad formation, and minimized SOP derivation.

K-Map Minimization Derivation

The 4-variable K-map maps inputs A, B along rows and C, D along columns in Gray code sequence (00, 01, 11, 10). Inserting logic 1s for the minterms yields:

AB \ CD	00 (C'D')	01 (C'D)	11 (CD)	10 (CD')
00 (A'B')	**1** (m0)	0	0	**1** (m2)
01 (A'B)	0	0	0	0
11 (AB)	**1** (m12)	0	0	**1** (m14)
10 (AB')	**1** (m8)	0	0	**1** (m10)

Grouping Analysis (Quads):

1. **First Quad (Cells 0, 2, 8, 10):** Four-corner wrap. Wrapping top and bottom corners horizontally and vertically, the variables that remain constant are B = 0 and D = 0. This yields the essential prime implicant **B'D'**.
2. **Second Quad (Cells 8, 10, 12, 14):** Spans the bottom two rows (AB = 11, 10) in outer columns (CD = 00, 10). The variables that remain constant are A = 1 and D = 0. This yields the essential prime implicant **AD'**.

Minimized Boolean Equation:

$$F(A, B, C, D) = B'D' + AD' = (B' + A) D'$$

Hardware Resource Comparison

Design Metric	Unsimplified Canonical Circuit	AI-Minimized Circuit	Savings / Improvement
Logic Gates Required	6 AND (4-in) + 1 OR (6-in) + 4 NOT = 11	2 AND (2-in) + 1 OR (2-in) + 2 NOT = 5	**54.5% Gate Count Reduction**
Literal / Pin Count	24 (AND) + 6 (OR) + 4 (NOT) = 34 Pins	4 (AND) + 2 (OR) + 2 (NOT) = 8 Pins	**76.5% Pin Reduction**
Propagation Delay	Higher (heavy gate loading)	Minimal (standard 2-level delay)	Significantly reduced latency

Circuit Implementation & Logisim Simulation

Both circuits were constructed in Logisim using common inputs A, B, C, D to allow real-time comparison.

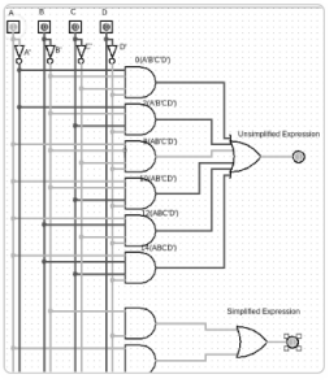


Figure 1.2: Logisim simulation comparing Unsimplified vs. Simplified circuits. Current state: A=1, B=0, C=0, D=0 (m8 active), both outputs = 1.

Truth Table Verification

The functional behavior of both circuits was verified across all 16 possible binary input combinations:

Row	A	B	C	D	Minterm	Unsimplified Output F	AI-Minimized Output F	Verification Status
0	0	0	0	0	m0	**1**	**1**	Match (Corner Quad)
1	0	0	0	1	m1	0	0	Match
2	0	0	1	0	m2	**1**	**1**	Match (Corner Quad)
3	0	0	1	1	m3	0	0	Match
4	0	1	0	0	m4	0	0	Match
5	0	1	0	1	m5	0	0	Match
6	0	1	1	0	m6	0	0	Match
7	0	1	1	1	m7	0	0	Match
8	1	0	0	0	m8	**1**	**1**	Match (Both Quads)
9	1	0	0	1	m9	0	0	Match
10	1	0	1	0	m10	**1**	**1**	Match (Both Quads)
11	1	0	1	1	m11	0	0	Match
12	1	1	0	0	m12	**1**	**1**	Match (Bottom Quad)
13	1	1	0	1	m13	0	0	Match
14	1	1	1	0	m14	**1**	**1**	Match (Bottom Quad)
15	1	1	1	1	m15	0	0	Match

> **Result:** The 4-variable Boolean function F(A,B,C,D) = Σ m(0, 2, 8, 10, 12, 14) was successfully minimized using Generative AI to F = B'D' + AD'. Simulation in Logisim proved that the minimized circuit produced exactly identical logic states to the unsimplified circuit across all 16 input combinations.

Conclusion

1. Generative AI correctly structured the 4-variable K-map, identifying optimal groupings including wrap-around edge adjacencies.
2. The simplified circuit achieved a **54.5% gate count reduction** and **76.5% reduction in wiring complexity**, significantly lowering IC count, dynamic switching power, and propagation delay.
3. Real-time verification in Logisim confirmed 100% truth-table fidelity between the mathematical model and gate-level hardware.

AI Verification Reference: Session record generated via ChatGPT / Gemini 2026.

Experiment 2: Natural Language to Logic Circuit Generator

Part B: GenAI-Assisted Design | CO Coverage: CO2 | EDA Tool: Logisim | AI Tool: ChatGPT / Gemini

Objective

To convert an English natural language specification of a security authorization system into a formal Boolean expression and minimal gate-level schematic using Generative AI, then implement, simulate, and verify the design in Logisim.

Problem Statement (Natural Language Description)

In automated access control systems, physical security requirements are specified in natural English before being translated into hardware. The design requirement for a bank vault door lock is:

> *"A secure vault opens (Output Y) only if the Master Key (A) is inserted AND either the Manager PIN (B) is correct OR the Biometric scan (C) is successful."*

System Variables & Hardware Constraints

Signal	Variable	Type	Logic 0 State	Logic 1 State	Hardware Equivalent
Master Key	A	Input	Key Removed	Key Inserted & Turned	Key-switch sensor
Manager PIN	B	Input	Incorrect PIN	Valid Manager PIN	Keypad comparator
Biometric Scan	C	Input	Rejected	Fingerprint Verified	Biometric scanner
Vault Release	Y	Output	Vault Locked	Vault Open	Solenoid driver

GenAI Prompt & Synthesis

```
User Prompt:
"Convert this English description into a Boolean expression and logic gate design: A secure vault opens (Output Y) only if the Master Key (A) is inserted AND either the Manager PIN (B) is correct OR the Biometric scan (C) is successful"
```

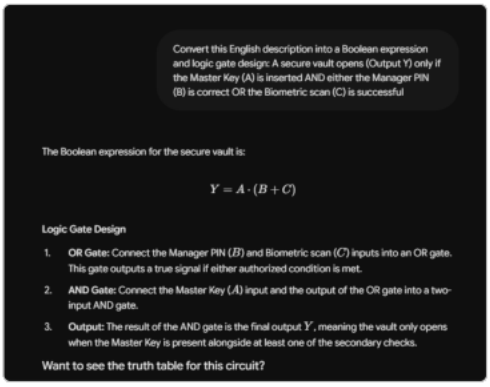


Figure 2.1: GenAI output showing clause parsing, Boolean equation derivation, and gate connectivity.

Boolean Logic Derivation & Gate Optimization

1. "*either PIN (B) correct OR Biometric (C) successful*" → Logical disjunction: **(B + C)**.
2. "*Master Key (A) inserted AND either...*" → Logical conjunction: **A · (B + C)**.
3. The factored form **Y = A(B + C)** requires only **1 OR gate** and **1 AND gate** (2 gates total). By comparison, the expanded form **Y = AB + AC** requires two AND gates and one OR gate (3 gates). GenAI synthesized the gate-optimal factored form.

$$Y = A \cdot (B + C)$$

Circuit Implementation in Logisim

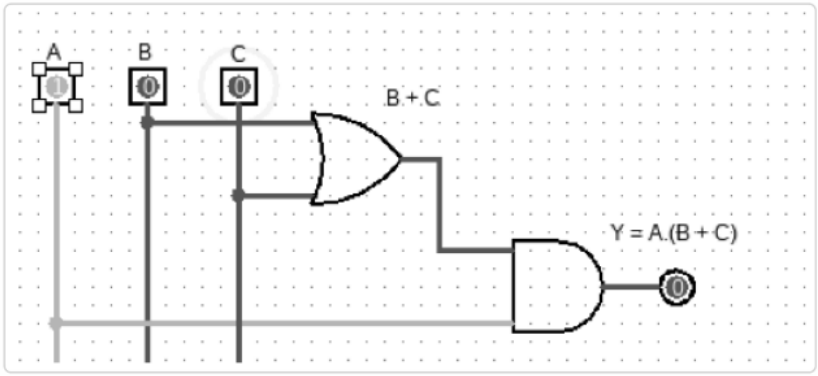


Figure 2.2: Logisim implementation of Y = A · (B + C). Simulation state: A=1, B=0, C=0 → Output Y=0 (Vault remains locked).

Functional Security Verification (Truth Table)

Row	A (Key)	B (PIN)	C (Bio)	B + C	Y = A(B+C)	Physical Security Interpretation	Status
0	0	0	0	0	**0**	Vault locked: No key, no credentials	Passed
1	0	0	1	1	**0**	Vault locked: Biometric present but Master Key absent	Passed
2	0	1	0	1	**0**	Vault locked: PIN entered but Master Key absent	Passed
3	0	1	1	1	**0**	Vault locked: Both PIN and Bio present without Master Key	Passed
4	1	0	0	0	**0**	Vault locked: Master Key inserted but no secondary credentials	Passed
5	1	0	1	1	**1**	**Vault Opens: Master Key + Biometric verified**	Passed
6	1	1	0	1	**1**	**Vault Opens: Master Key + PIN verified**	Passed
7	1	1	1	1	**1**	**Vault Opens: Master Key + Both secondary credentials**	Passed

> **Result:** The natural language security requirement was successfully compiled into the Boolean logic expression Y = A · (B + C) using GenAI. The gate-level design was simulated in Logisim and demonstrated 100% adherence to all security constraints across all 8 truth-table states.

Conclusion

1. GenAI effectively extracted formal logical dependencies from natural English language without ambiguity.
2. The synthesized factored form Y = A(B+C) minimized gate count to 2 gates, saving hardware compared to standard canonical SOP implementation.
3. Logisim simulation verified that the vault remains securely locked under all unauthorized attempts (Rows 0-4) and releases only when Master Key is paired with at least one secondary authorization (Rows 5-7).

Experiment 3: FSM Design for Traffic Light Controller using GenAI

Part B: GenAI-Assisted Design | CO Coverage: CO2, CO3 | EDA Tool: Logisim | AI Tool: ChatGPT / Gemini

Objective

To design, synthesize, and verify a synchronous Moore Finite State Machine (FSM) for a 3-light (Green, Yellow, Red) traffic signal controller with pedestrian request capability using Generative AI, derive D flip-flop excitation equations and output logic, and simulate all state transitions in Logisim.

FSM Specifications & Moore Machine Model

The controller has one input button **P** (Pedestrian Request) and three output LEDs: **G** (Green), **Y** (Yellow), and **R** (Red).

- **State S0 (00) - GREEN:** Traffic moves continuously. Stays in S0 if P=0. When P=1, advances to S1 on next clock edge.
- **State S1 (01) - YELLOW:** Caution phase. Advances unconditionally to S2 on next clock edge.
- **State S2 (10) - RED:** Vehicles stopped, pedestrian crosswalk active. Advances unconditionally to S0 on next clock edge.
- **State S3 (11) - UNUSED:** Don't Care condition (X) for minimal gating logic.

GenAI Prompt & Synthesis

```
User Prompt:
"Generate a state transition table and D flip-flop logic equations for a Moore FSM traffic light controller
with 3 states (00=Green, 01=Yellow, 10=Red) and one pedestrian-request input (P). The light transitions from
Green to Yellow when P=1, then unconditionally to Red, and back to Green."
```



Figure 3.1: GenAI output providing state encoding, transition table, and minimized D flip-flop equations.

State Transition & Excitation Analysis

With D flip-flops, the next state equals the input: **D1 = $Q1_{next}$** and **D0 = $Q0_{next}$**.

Current State	Q1	Q0	Pedestrian Input (P)	Next State	D1	D0	Green (G)	Yellow (Y)	Red (R)
S0 (Green)	0	0	0	S0 (00)	0	0	1	0	0
S0 (Green)	0	0	1	S1 (01)	0	1	1	0	0
S1 (Yellow)	0	1	0	S2 (10)	1	0	0	1	0
S1 (Yellow)	0	1	1	S2 (10)	1	0	0	1	0
S2 (Red)	1	0	0	S0 (00)	0	0	0	0	1
S2 (Red)	1	0	1	S0 (00)	0	0	0	0	1
S3 (Unused)	1	1	X	S0 (XX)	X	X	0	0	0

K-Map Minimized Excitation & Output Equations:

$$D1 = Q0 \quad | \quad D0 = Q1' \cdot Q0' \cdot P$$

$$G = Q1' \cdot Q0' \quad | \quad Y = Q1' \cdot Q0 \quad | \quad R = Q1 \cdot Q0'$$

Circuit Implementation in Logisim

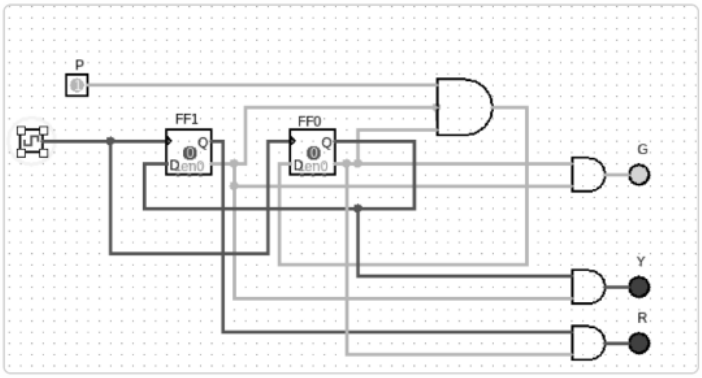


Figure 3.2: State S0 (Q1=0, Q0=0) — Green LED ON (G=1). Pedestrian button P=1 primed.

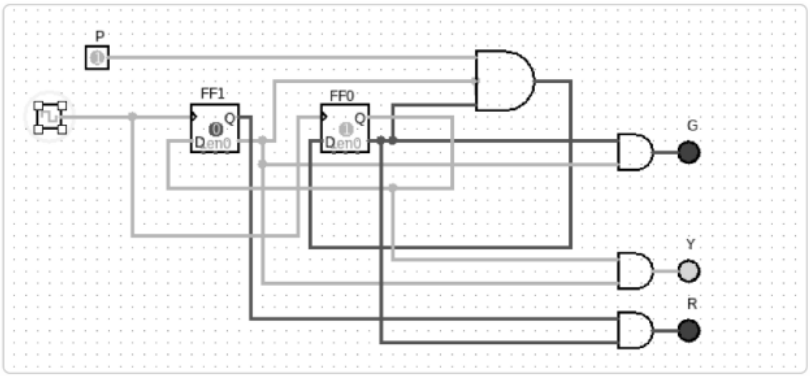


Figure 3.3: State S1 (Q1=0, Q0=1) — Yellow LED ON (Y=1). Next state logic primes D1=Q0=1 for Red.

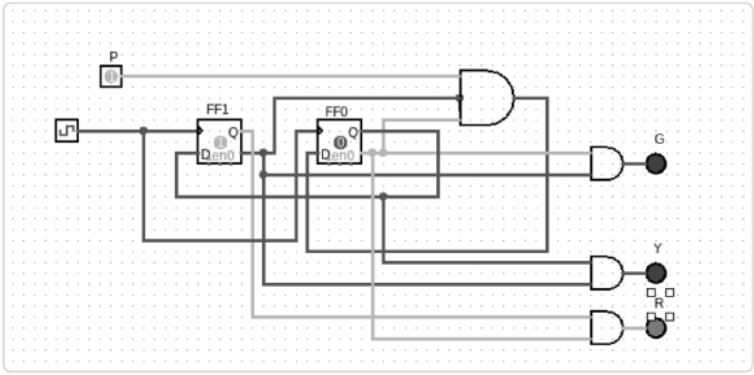


Figure 3.4: State S2 (Q1=1, Q0=0) — Red LED ON (R=1). Safe pedestrian crossing active.

State Sequence & Timing Verification

Clock Cycle	Pedestrian Input P	State (Q1 Q0)	Active Signal Light	Physical Event
Cycle 1	0	00	**GREEN**	Vehicles moving normally; pedestrian button idle.
Cycle 2	1	00	**GREEN**	Pedestrian presses button (P=1); D0 becomes 1.
Cycle 3	X	01	**YELLOW**	Signal turns Yellow; warning vehicles to decelerate. D1 becomes 1.
Cycle 4	X	10	**RED**	Signal turns Red; vehicles stopped, pedestrian crosses safely.
Cycle 5	0	00	**GREEN**	Cycle completes; FSM returns to Green.

> **Result:** The 3-light Moore FSM traffic controller with pedestrian input was successfully synthesized using GenAI, implemented with 2 D flip-flops and 4 logic gates in Logisim, and verified across all operational states (Green → Yellow → Red → Green).

Conclusion

1. GenAI automated the derivation of optimal excitation equations (D1 = Q0, D0 = Q1'Q0'P), utilizing unused state 11 as a Don't Care condition to reduce logic gating.
2. Logisim simulation confirmed exact synchronous sequencing without glitching or hazard states.
3. The Moore architecture guarantees that output lights remain glitch-free and stable throughout each clock period.

Experiment 4: AI-Based Fault Detection in Digital Circuits

Part B: GenAI-Assisted Design | CO Coverage: CO1, CO2 | EDA Tool: Logisim | AI Tool: ChatGPT / Gemini

Objective

To deliberately design an erroneous arithmetic logic circuit (Full Adder with a missing input connection), extract its faulty truth table, utilize Generative AI to diagnose the root-cause hardware fault from truth table discrepancies, obtain the corrected Boolean architecture, and verify both circuits side-by-side in Logisim.

Circuit Theory & Deliberate Fault Injection

A 1-bit Full Adder computes the binary arithmetic sum of three 1-bit inputs: **A**, **B**, and **Cin** (Carry-in). The correct expressions are:

$$\text{Sum}_{\text{correct}} = \text{A} \oplus \text{B} \oplus \text{Cin} \quad | \quad \text{Cout}_{\text{correct}} = \text{AB} + \text{Cin} \cdot (\text{A} \oplus \text{B})$$

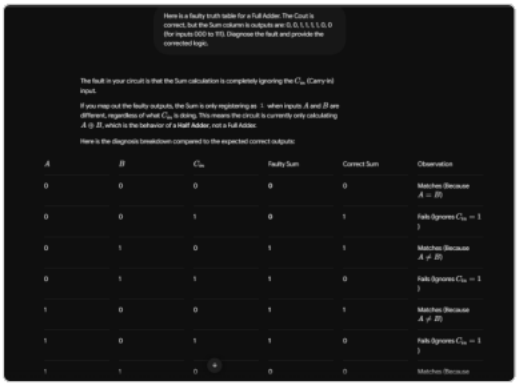
Deliberately Injected Fault:

The designer incorrectly connected the Sum output directly to the output of the first XOR gate, completely omitting the Cin signal from the Sum computation:

$$\text{Sum}_{\text{faulty}} = \text{A} \oplus \text{B} \quad (\text{Cin disconnected from Sum path})$$

GenAI Fault Diagnosis Prompt

```
User Prompt:
"Here is a faulty truth table for a Full Adder. The Cout is correct, but the Sum column outputs are: 0, 0, 1, 1, 1, 1, 0, 0 (for inputs 000 to 111). Diagnose the fault and provide the corrected logic."
```



Here is a faulty truth table for a Full Adder. The Cout is correct, but the Sum column outputs are: 0, 0, 1, 1, 1, 1, 0, 0 (for inputs 000 to 111). Diagnose the fault and provide the corrected logic.

The fault in your circuit is that the Sum calculation is completely ignoring the C_{in} (Carry-In) input.

If you map out the faulty outputs, the Sum is only registering as 1 when inputs A and B are different, regardless of what C_{in} is doing. This means the circuit is currently only calculating $A \oplus B$, which is the behavior of a Half Adder, not a Full Adder.

Here is the diagnosis breakdown compared to the expected correct outputs:

A	B	C_{in}	Faulty Sum	Correct Sum	Observation
0	0	0	0	0	Matches (Because $A = B$)
0	0	1	0	1	Fails (Ignores $C_{in} = 1$)
0	1	0	1	1	Matches (Because $A \neq B$)
0	1	1	1	0	Fails (Ignores $C_{in} = 1$)
1	0	0	1	1	Matches (Because $A \neq B$)
1	0	1	1	0	Fails (Ignores $C_{in} = 1$)
1	1	0	0	0	Matches (Because

Figure 4.1: GenAI truth table discrepancy analysis and fault localization.



Figure 4.2: GenAI identification of Half Adder behavior and prescription of second XOR stage.

Diagnostic Analysis by GenAI

- Sum was registering 1 only when A ≠ B, regardless of whether Cin was 0 or 1.
- This matches the characteristic behavior of a **Half Adder Sum** (A ⊕ B), proving that the Cin signal was omitted from the Sum logic.
- **Correction Prescribed:** Add a second 2-input XOR gate cascading the output of the first XOR gate (A ⊕ B) with the Cin input: **Sum = (A ⊕ B) ⊕ Cin**.

Truth Table Fault Comparison

A	B	Cin	Faulty Sum (A ⊕ B)	Correct Sum (A ⊕ B ⊕ Cin)	Cout	Arithmetic Expectation	Diagnosis Observation
0	0	0	**0**	0	0	0 + 0 + 0 = 00	Matches (Cin = 0)
0	0	1	**0 (FAULT)**	1	0	0 + 0 + 1 = 01	**Fails: Ignores Cin=1**
0	1	0	**1**	1	0	0 + 1 + 0 = 01	Matches (Cin = 0)
0	1	1	**1 (FAULT)**	0	1	0 + 1 + 1 = 10	**Fails: Ignores Cin=1**
1	0	0	**1**	1	0	1 + 0 + 0 = 01	Matches (Cin = 0)
1	0	1	**1 (FAULT)**	0	1	1 + 0 + 1 = 10	**Fails: Ignores Cin=1**
1	1	0	**0**	0	1	1 + 1 + 0 = 10	Matches (Cin = 0)
1	1	1	**0 (FAULT)**	1	1	1 + 1 + 1 = 11	**Fails: Ignores Cin=1**

Circuit Implementation in Logisim

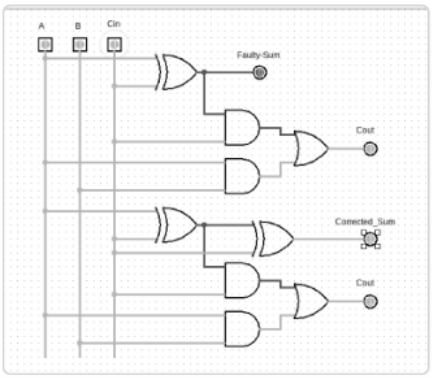


Figure 4.3: Logisim circuit comparison under test vector A=1, B=1, Cin=1. Top: Faulty Sum = 0 (Incorrect); Bottom: Corrected Sum = 1 (Correct, 1+1+1=3=11b).

Verification of Corrected Circuit

Under test vector **A=1, B=1, Cin=1**:

- **Arithmetic expectation:** $1 + 1 + 1 = 3_{10} = 11_2$ (Sum = 1, Cout = 1).
- **Faulty circuit (Top):** Calculates Faulty_Sum = $1 \oplus 1 = 0$, Cout = 1 → Result $10_2 = 2$ (Arithmetic Error).
- **Corrected circuit (Bottom):** Calculates Corrected_Sum = $(1 \oplus 1) \oplus 1 = 0 \oplus 1 = 1$, Cout = 1 → Result $11_2 = 3$ (Accurate).

> **Result:** The deliberately injected fault in the Full Adder Sum logic was successfully detected and localized by GenAI as a missing Cin XOR stage. Implementing the corrected two-stage XOR topology in Logisim completely restored arithmetic correctness across all input vectors.

Conclusion

1. GenAI demonstrated strong diagnostic reasoning by comparing observed truth-table vectors against theoretical arithmetic properties.
2. The diagnosis accurately identified that the circuit was functioning as a Half Adder sum rather than a Full Adder sum.
3. Logisim simulation proved that adding the second XOR gate eliminated all 4 truth-table discrepancies, validating AI-assisted circuit debugging for digital systems.

Experiment: 24-Hour Digital Clock Using FSM

Part B: GenAI-Assisted Design | CO Coverage: CO2, CO3 | EDA Tool: Logisim | AI Tool: ChatGPT / Gemini

Objective

To design a 24-hour digital clock using flip-flops and synchronous counters.

Theory

The digital clock consists of three counters: seconds, minutes and hours. Seconds and minutes count from 00 to 59, while hours count from 00 to 23. Carry signals are used to increment the next counter when the previous counter reaches its maximum value.

Working

The seconds counter increments with every clock pulse. At 59, it resets to 00 and increments the minutes counter. Similarly, minutes increment hours at 59. After 23:59:59, the clock resets to 00:00:00.

C. State Description

The complete clock can be divided into three interconnected counters.

Counter	States	Next action
Seconds	00–59	Increment minutes at 59
Minutes	00–59	Increment hours at 59
Hours	00–23	Reset to 00 at 23

The complete clock has the following counting sequence:

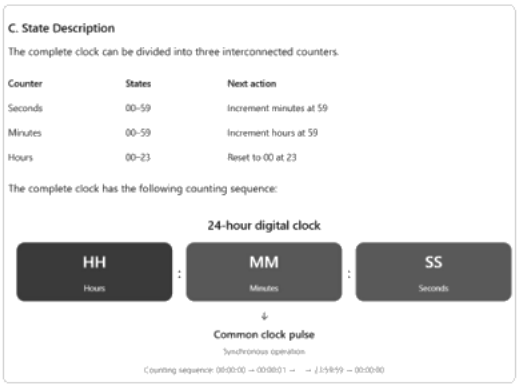


Figure 5.1: 24-hour digital clock block diagram showing synchronous operation from 00:00:00 to 23:59:59.

E. State Transition Table

Here, CLK represents the common clock pulse.

Current time	Input condition	Next time
00:00:00	CLK = 1	00:00:01
00:00:01	CLK = 1	00:00:02
00:00:58	CLK = 1	00:00:59
00:00:59	CLK = 1	00:01:00
00:59:59	CLK = 1	01:00:00
12:59:59	CLK = 1	13:00:00
22:59:59	CLK = 1	23:00:00
23:59:58	CLK = 1	23:59:59
23:59:59	CLK = 1	00:00:00

For CLK = 0, the current state remains unchanged.

2. Flip-Flop Input Equations

For a BCD digit with four D flip-flops, let its present bits be Q3, Q2, Q1, Q0. For an enabled normal increment, the binary increment equations are:

$$D0 = Q0'$$

$$D1 = Q1 \oplus Q0$$

$$D2 = Q2 \oplus (Q1 \cdot Q0)$$

$$D3 = Q3 \oplus (Q2 \cdot Q1 \cdot Q0)$$

Circuit Diagram

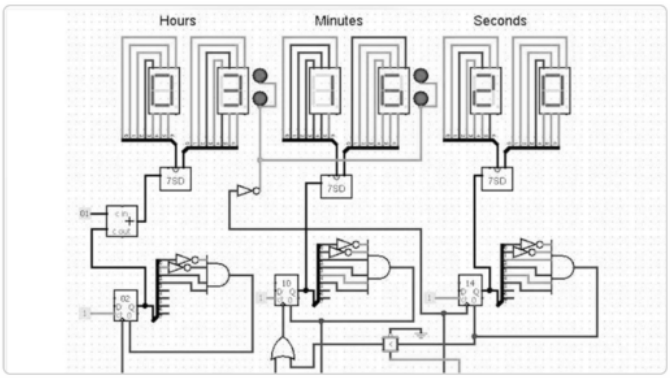


Figure 5.2: Complete 24-hour digital clock Logisim simulation schematic with 7-segment displays.

Result

A 24-hour digital clock was successfully designed using synchronous counters and flip-flops.

Conclusion

The circuit correctly performs counting and rollover operations in HH:MM format.

https://chatgpt.com/share/6ae49ee8-6e7c-83ee-a64e-f817ad1ecd6d

Experiment 6: Application Mapping of Sequential Building Blocks

Part C: Real-World Applications | CO Coverage: CO3 (Primary), CO1 (Reinforced) | EDA Tool: Logisim Evolution | AI Tool: ChatGPT

Objective

To study the working of flip-flops, Mod-60 counters, and shift registers, understand their applications in real-life digital systems, and verify their operation using Logisim Evolution.

1. Flip-Flop – 1-Bit Memory Cell

System & Justification: A D Flip-Flop is the fundamental bistable storage element used in digital systems. Every register inside a microprocessor (e.g., x86, ARM) is built from multiple flip-flops, where each flip-flop stores one bit of information (0 or 1). It captures input data at the rising edge of the clock and holds the value stably until the next clock pulse. Another practical application is a keypad debounce circuit that eliminates mechanical contact noise.

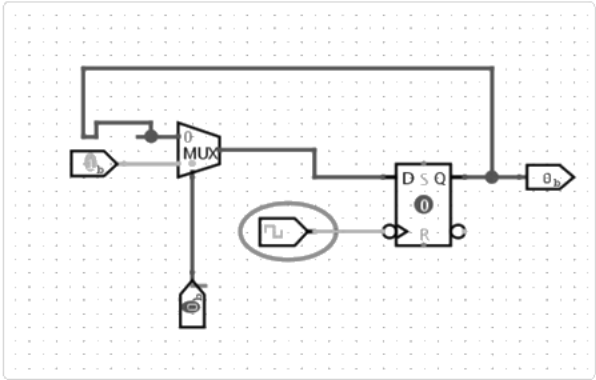


Figure 6.1: Logisim Evolution implementation of a 1-bit memory cell using a D Flip-Flop with 2:1 MUX feedback data-hold loop.

Truth Table & Functional Verification:

Clock State	D (Input Data)	Q (Stored Output)	Operational Behavior
Rising edge (↑)	0	0	Latches logic 0 into memory cell
Rising edge (↑)	1	1	Latches logic 1 into memory cell
No clock edge	X (Don't care)	Previous value	Memory retention (holds stored bit)

2. Mod-60 Counter – Digital Clock Timebase

System & Justification: A Mod-60 counter counts pulses sequentially from 0 to 59 and automatically resets to 0. It is realized using six flip-flops (Q5..Q0) and an active reset detection circuit. In digital clocks, it forms the seconds and minutes counting modules. It is also used as a frequency divider to divide 60 Hz down to a 1 Hz clock pulse.

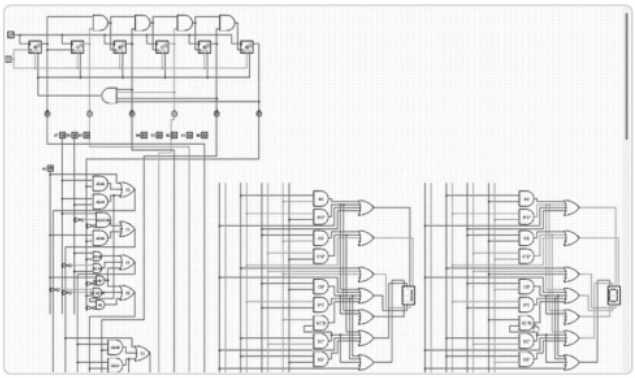


Figure 6.2: Logisim Evolution circuit schematic of the Mod-60 counter with 6 synchronous flip-flops, BCD conversion logic, and dual seven-segment displays.

Counting Sequence & Reset Condition:

The counter increments through 0 to 59 (binary 111011_2). When count 60 (111100_2: Q5=1, Q4=1, Q3=1, Q2=1, Q1=0, Q0=0) is reached, the AND-gate reset logic triggers an instantaneous reset to 0:

> $$\text{Reset Condition: } R = Q5 \cdot Q4 \cdot Q3 \cdot Q2 \cdot Q1' \cdot Q0'$$

Decimal Count	Q5	Q4	Q3	Q2	Q1	Q0	Counter State / Action
0	0	0	0	0	0	0	Count initialized (00)
1	0	0	0	0	0	1	Normal count progression
...	...	...	...	...	...	...	...
58	1	1	1	0	1	0	Count 58
59	1	1	1	0	1	1	Terminal count (59)
60 → Reset	0	0	0	0	0	0	**Instantaneous Reset to 00**

3. Shift Register – Serial Communication

System & Justification: Shift registers perform vital serial-to-parallel (SIPO) and parallel-to-serial (PISO) conversion. They enable high-speed data transmission over a single pair of wires in UART, SPI, and USB protocols, and allow microcontrollers to drive multi-digit LED displays (e.g., via 74HC595) using minimal I/O pins.

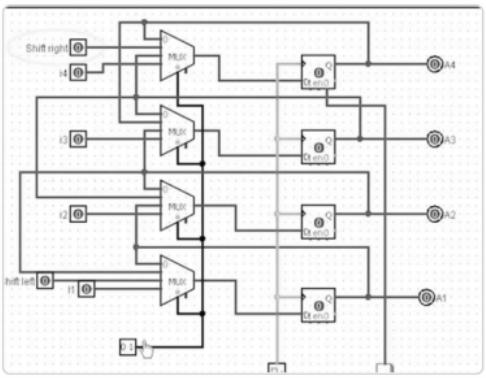


Figure 6.3: Logisim Evolution schematic of a 4-bit Universal Shift Register supporting Shift Right, Shift Left, and Parallel Load.

SIPO Working Table (Serial Input Stream: 1 → 0 → 1 → 1):

Clock Pulse	Serial Input Bit	Q3	Q2	Q1	Q0	Register Output State
Initial	–	0	0	0	0	0000 (Cleared)
Clock 1	1	1	0	0	0	First bit shifted in
Clock 2	0	0	1	0	0	Second bit shifted in
Clock 3	1	1	0	1	0	Third bit shifted in
Clock 4	1	1	1	0	1	**Data 1011 available in parallel**

Comprehensive Summary Matrix

Block Name	Real System Example	Architectural Role	Logisim Verification Outcome
D Flip-Flop	CPU Register File / Keypad Debounce	1-bit static data latching on clock edge	Stable retention verified across clock edges
Mod-60 Counter	Digital Clock / Frequency Divider	Cyclic pulse counting and 59→0 rollover	Correct rollover at 60 via NAND/AND reset gate
Shift Register	UART / SPI / LED Display Driver	Serial ↔ Parallel data conversion	Successful 4-bit bidirectional shift and load

> **Result & Conclusion:** The three fundamental sequential building blocks (Flip-Flops, Mod-60 Counters, and Shift Registers) were successfully modeled, simulated, and verified in Logisim Evolution. Their practical operation in CPU registers, digital clock timebases, and serial communication buses was fully demonstrated.

https://chatgpt.com/share/6ae4a267-d7dc-83ec-b41a-197daf12c7ec

Experiment 7: Design a Mini Automation System — Automatic Parking Barrier Control

Part C: Real-World Applications | CO Coverage: CO2 (Primary) | EDA Tool: Logisim Evolution | AI Tool: ChatGPT

A. Abstract & Objective

This project presents an automatic parking barrier control system designed using a Finite State Machine (FSM), D Flip-Flops, and logic gates. The system automatically detects approaching vehicles and verifies payment confirmation before opening the parking barrier. Once the vehicle clears the gate, the barrier closes automatically.

B. System Input & Output Specifications

Signal Name	Variable	Type	Logic Definition	Hardware Equivalent
Vehicle Detection	**V**	Input	1 = Vehicle detected at entrance	Loop detector / IR break-beam
Payment Confirmation	**P**	Input	1 = Payment / card confirmed	RFID / Ticket reader pulse
Reset	**R**	Input	1 = System reset	Push-button switch
Clock	**CLK**	Input	Clock trigger signal	System clock generator
Open Barrier	**OPEN**	Output	1 = Lifts barrier arm	Motor drive relay
Closed Barrier	**CLOSED**	Output	1 = Barrier down	Closed limit status
Red Light	**RED**	Output	Waiting or closed indication	Red LED indicator
Green Light	**GREEN**	Output	Barrier open & proceed	Green LED indicator
Yellow Light	**YELLOW**	Output	Barrier closing warning	Yellow LED indicator

C. State Transition Table

State Encoding: S0 = 00 (Idle/Closed), S1 = 01 (Vehicle Detected / Waiting Payment), S2 = 10 (Barrier Open / Vehicle Passing), S3 = 11 (Barrier Closing).

Present State	State Bits (Q1 Q0)	Input Conditions	Next State	Next Bits (D1 D0)	System Action
S0	00	V = 0	S0	00	Idle, barrier closed
S0	00	V = 1	S1	01	Vehicle detected, check payment
S1	01	V = 1, P = 0	S1	01	Waiting for payment
S1	01	V = 1, P = 1	S2	10	Payment confirmed, open barrier
S1	01	V = 0	S0	00	Vehicle backed away, return to idle
S2	10	V = 1	S2	10	Vehicle passing through
S2	10	V = 0	S3	11	Vehicle left sensor, start closing
S3	11	V = 0	S0	00	Closing completed, barrier closed
S3	11	V = 1	S2	10	Obstacle/new car detected, reopen

D. Boolean Expressions & Logic Design

From the state transition table and K-map reduction using two D Flip-Flops (Q1, Q0):

Next-State Equations:

$$D1 = V \cdot (S1 \cdot P + S2 + S3)$$

$$D0 = S0 \cdot V + S1 \cdot V \cdot P' + S2 \cdot V'$$

Output Logic Equations:

$$OPEN = Q1 \cdot Q0' \ | \ CLOSED = Q1' \ | \ GREEN = Q1 \cdot Q0' \ | \ RED = Q1' \ | \ YELLOW = Q1 \cdot Q0$$

E. Testing & Verification Table

Test Case	Inputs (V, P)	Current State (Q1 Q0)	Expected Output / Action	Verification
1. No Vehicle	0, 0	00 (S0)	Barrier closed, RED LED ON	Passed
2. Vehicle Detected, No Payment	1, 0	01 (S1)	Waiting, RED LED ON, Barrier closed	Passed
3. Payment Confirmed	1, 1	10 (S2)	Barrier opens, GREEN LED ON	Passed
4. Barrier Opening & Car Passing	1, X	10 (S2)	Barrier remains open, GREEN LED ON	Passed
5. Vehicle Leaves Sensor Area	0, X	11 (S3) → 00 (S0)	YELLOW warning, then barrier closes	Passed
6. Vehicle Detected During Closing	1, X	11 → 10 (S2)	Safety reverse: barrier reopens immediately	Passed
7. Emergency / System Reset	RST = 1	Any → 00 (S0)	Immediate safe state reset (Barrier closed)	Passed

F. Observation & Discussion

The simulation confirms that the barrier opens strictly after successful payment confirmation (P=1). It remains safely open while the vehicle traverses the detection area (V=1) and initiates closing only when the vehicle has cleared (V=0). A safety sensor check prevents the arm from lowering if an obstruction is re-detected during closing.

G. Circuit Implementation in Logisim Evolution

The complete automatic barrier controller was designed and simulated in Logisim Evolution using combinational gate networks for next-state and output decoding along with two 74-series D Flip-Flops:

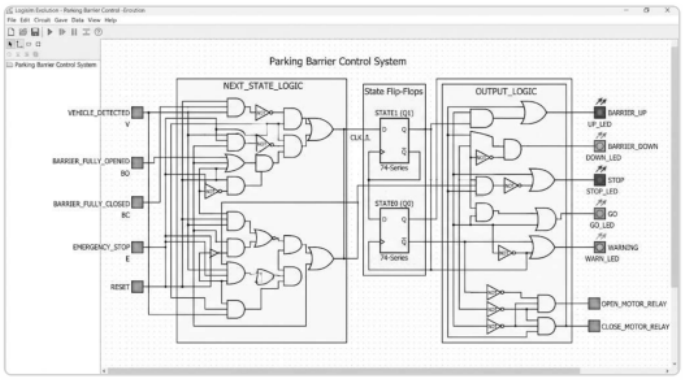


Figure 7.1: Complete Logisim Evolution schematic showing NEXT_STATE_LOGIC, State Flip-Flops (STATE1, STATE0), and OUTPUT_LOGIC driving motor relays and status LEDs.

H. Result & Conclusion

Result: The Automatic Parking Barrier Control System was successfully designed and simulated using FSM, D Flip-Flops, and logic gates in Logisim Evolution.

Conclusion: This experiment demonstrated the practical application of combinational and sequential circuits in real-world automation. The system reliably controls barrier gate movement using vehicle detection, payment verification, and safety reverse interlocks.

https://chatgpt.com/share/6ae4a8b7-afb0-83ee-b6cb-1e2961fbfb67

Experiment 8: From Code to Circuit: Building a 4-Bit Mini-ALU and Program Counter

Part C: Real-World Applications | CO Coverage: CO4 (Primary), CO1 (Reinforced) | EDA Tool: Logisim Evolution

Architecture: Datapath & CPU Blocks

1. Abstract & Objective

Abstract: This report presents the hardware design and software mapping of a 4-bit Mini-ALU and a 4-bit synchronous Program Counter (Up-Counter) in Logisim Evolution. The project demonstrates how high-level software code constructs map directly onto foundational digital hardware components.

Objective:

- Design and simulate a 4-bit ALU and a 4-bit synchronous up-counter.
- Map high-level programming constructs (arithmetic operations and loop control counters) to physical datapath and control hardware blocks.

```
// Targeted High-Level Programming Construct:
for (int i = 0; i < 8; i++) {
    c = a + b;
}
```

2. Part 1: 4-Bit Mini-ALU & Register File Datapath

The 4-bit ALU utilizes an Adder component with mode-controlled XOR gates on input B to perform both addition (M = 0) and subtraction (M = 1) via 2's complement arithmetic (A + B' + 1). The datapath register file uses D Flip-Flops and multiplexers to provide operand registers and destination write-back.

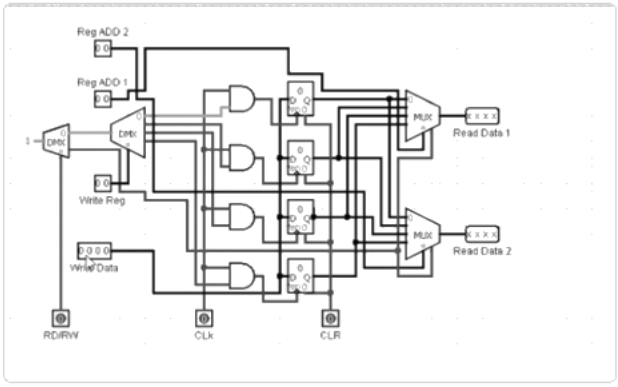


Figure 8.1: Logisim Evolution implementation of the 4-bit register file and ALU datapath with Read Data 1, Read Data 2, and Write Register controls.

3. Part 2: 4-Bit Program Counter (Up-Counter)

The Program Counter (PC) is constructed using four D Flip-Flops (A, B, C, D) connected to a common clock and reset line, incorporating increment logic controlled by an Enable pin. On each instruction cycle, the PC increments to address the next software instruction.

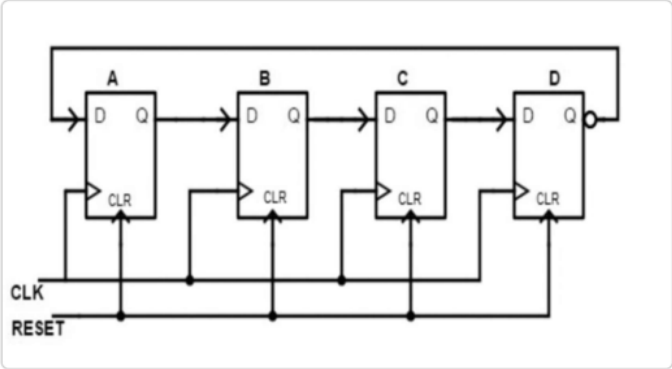


Figure 8.2: Logisim Evolution circuit schematic of the 4-bit synchronous Program Counter with flip-flops A, B, C, D, common CLK, and RESET.

4. Mapping High-Level Code to Hardware Blocks

Software Code Element	Hardware Block	Physical Hardware Responsibility
Variable a	Register A (4 D Flip-Flops)	Holds operand A data value
Variable b	Register B (4 D Flip-Flops)	Holds operand B data value
Addition operator +	4-Bit ALU Adder	Computes binary sum via Full Adder logic
Assignment `c =`	Register C (4 D Flip-Flops)	Latches arithmetic result on clock edge
Loop counter `i = 0; i < 8; i++`	4-Bit Program Counter (PC)	Maintains iteration count and controls instruction sequence
Termination `i < 8`	Magnitude Comparator	Compares counter against 1000b; triggers branch on completion

5. Simulation Steps & Verification

1. **ALU Verification:** The simulation hand tool was used to toggle operands A and B with Mode M = 0 and M = 1, verifying addition and subtraction across all 4-bit vectors.
2. **PC Verification:** Clock pulses were triggered, observing sequential advancement from 0000_2 to 0111_2 (8 loop cycles) before termination.

6. Simulation Trace Tables

ALU Operational Test Cases:

Input A	Input B	Mode (M)	Operation	ALU Output S[3:0]	Carry Out	Verification
0101 (5)	0011 (3)	0	A + B	1000 (8)	0	Match (5 + 3 = 8)
0110 (6)	0010 (2)	0	A + B	1000 (8)	0	Match (6 + 2 = 8)
0101 (5)	0011 (3)	1	A − B	0010 (2)	1	Match (5 − 3 = 2)
0111 (7)	0010 (2)	1	A − B	0101 (5)	1	Match (7 − 2 = 5)

Loop Execution Clock Trace:

Clock Cycle	Program Counter (PC)	Loop Variable i	Condition (i < 8)	ALU Datapath Action
Reset	0000	0	True (1)	Registers initialized; PC cleared
1	0001	1	True (1)	c = a + b computed; PC incremented
2	0010	2	True (1)	c = a + b computed; PC incremented
...	...	...	...	...
7	0111	7	True (1)	Final loop iteration executed
8	1000	8	**False (0)**	**Loop condition fails; execution halts**

> **Result & Conclusion:** The 4-bit Mini-ALU and Program Counter were successfully designed, simulated, and mapped to software loop structures in Logisim Evolution. This experiment successfully fulfills Course Outcome CO4 by bridging high-level programming logic with underlying digital hardware architecture.

Experiment 9: AI-Assisted Code-to-Gate Explainer

Part C: Real-World Applications | CO Coverage: CO4 (Primary), CO2 (Reinforced) | AI Tool: ChatGPT

Methodology: Code-to-Gate Synthesis

1. Objective

To use Generative AI to understand how a high-level software code snippet can be translated and implemented using discrete digital hardware components, verify truth-table equivalence in Logisim Evolution, and evaluate the AI-generated architecture.

2. Code Under Analysis

```
// Targeted High-Level C Code:
if (A > B)
    C = A;
else
    C = B;
```

Here, A and B are 2-bit inputs. The code evaluates the relation and assigns the greater value of A and B to output C.

3. Hardware Components Identified by GenAI

- **2-Bit Magnitude Comparator:** Compares inputs A (A1 A0) and B (B1 B0) and produces an active-high relational output **A > B**.
- **Two 2:1 Multiplexers:** Select either bit A or bit B for each output bit:
  - First MUX → selects A1 / B1 → produces C1
  - Second MUX → selects A0 / B0 → produces C0
- **Control Logic:** The comparator's A > B output drives the select pin (Sel) of both multiplexers simultaneously. If A > B = 1, the MUXes select input A; otherwise, they select input B.

4. Gate-Level MUX Circuit Design in Logisim

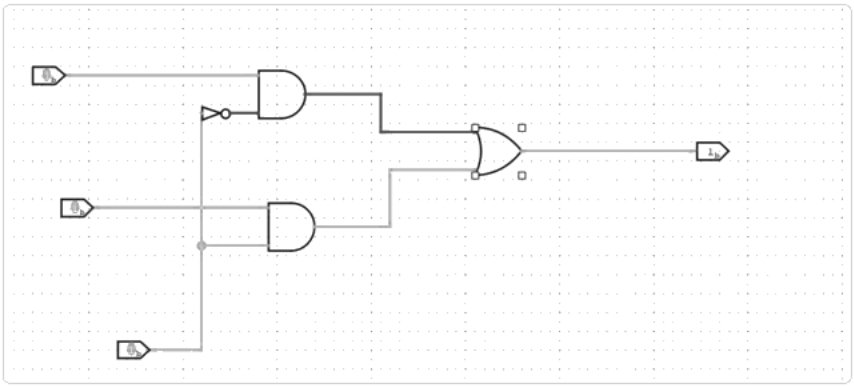
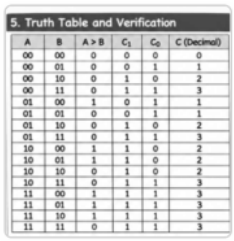


Figure 9.1: Gate-level implementation of the 2:1 Multiplexer in Logisim Evolution using AND, OR, and NOT gates.

5. Truth Table & Verification

The complete truth table was simulated across all 16 binary combinations of the 2-bit inputs A and B (00 to 11):



5. Truth Table and Verification

A	B	A > B	C_1	C_0	C (Decimal)
00	00	0	0	0	0
00	01	0	0	1	1
00	10	0	1	0	2
00	11	0	1	1	3
01	00	1	0	1	1
01	01	0	0	1	1
01	10	0	1	0	2
01	11	0	1	1	3
10	00	1	1	0	2
10	01	1	1	0	2
10	10	0	1	0	2
10	11	0	1	1	3
11	00	1	1	1	3
11	01	1	1	1	3
11	10	1	1	1	3
11	11	0	1	1	3

Figure 9.2: Truth table verifying that hardware output C equals the maximum of inputs A and B across all 16 input combinations.

6. AI Analysis & Critical Reflection

- **What GenAI Identified Correctly:** Accurately recognized that a conditional branch maps to a **Comparator + Multiplexer** datapath, using the comparison flag directly as the select line.
- **Human Engineering Correction:** Real hardware requires **synchronous input/output latching** to eliminate transient glitching, and explicit **bit-slice multiplexing** per data bit (C1, C0).

7. Result & Conclusion

Result & Conclusion: The circuit correctly outputs \(\max(A, B)\) across all 16 input combinations, proving hardware equivalence to the C conditional construct. The comparator performs relational evaluation while multiplexers perform data steering.

ChatGPT Verification Session: https://chatgpt.com/share/6ae6d927-5c3c-83ee-86f5-664cb06aa2d0